

WMS — Installation & Initial Setup Guide

Audience: a brand-new administrator with no prior knowledge of this project, Odoo, PostgreSQL, or warehouse setup. Follow this top to bottom and you will have a deployed, secured, backed-up, and operational Warehouse Management System.

What you are deploying: an Odoo 19 Community Edition WMS that runs **natively on Windows** (no Docker). It models storage as **Warehouse → Zone → Rack → Compartment → Slot**, scans goods in/out with barcodes, prints 4×1 inch thermal labels, tracks damage/repair/returns, forecasts demand offline, and ships an in-app **Training Academy**.

Conventions in this guide - Commands run in **Windows PowerShell**. "Admin PowerShell" = right-click PowerShell → **Run as administrator**. -  **CHECKPOINT** = verify this before moving on. -  **CAPTURE** = a screenshot worth taking for your runbook/training. - Paths assume the project lives at `D:\Udhay\projects\Inventory_mngt` — adjust to wherever you cloned it.

Source of truth for connection details: after install, the file `config\odoo.native.conf` holds the real database host/port/user/password and security flags. When in doubt, read that file.

Table of contents

- 1. System Requirements
- 2. Installation
- 3. Database Setup
- 4. First Launch
- 5. Windows Service
- 6. Security Setup
- 7. Warehouse Configuration
- 8. User Setup
- 9. Label Printing
- 10. Training
- 11. Product Onboarding
- 12. First Inventory Workflow
- 13. Backup & Recovery
- 14. Go-Live Checklist
- 15. Troubleshooting

1. System Requirements

Tier	CPU	RAM	Storage	OS	Python	PostgreSQL
Minimum (1–2 users, evaluation)	2 cores	4 GB	10 GB free	Windows 10 (64-bit)	3.12	16+
Recommended (small trust, 2–5 users)	4 cores	8 GB	25 GB free (SSD)	Windows 11 / Server 2022	3.12	16+
Production (always-on, off-site access)	4–8 cores	16 GB	50 GB+ SSD, second disk/share for backups	Windows 11 / Server 2022	3.12	16+

Also required - **PowerShell 5.1+** (built into Windows) and **administrator rights** (for the one-shot installer's `winget` packages and the Windows service). - **Internet access** for the first install (downloads PostgreSQL, Python, wkhtmltopdf, Git, and the Odoo 19 source ≈ a few hundred MB). - **Disk:** ~5 GB for the Odoo source + Python venv + database; add headroom for filestore (attachments/photos) and rotated backups. - **Hardware (optional, for the floor):** a USB/Bluetooth HID barcode scanner and a thermal label printer (tested: True-Ally 100×25 mm direct-thermal stock on a TSC TE244).

 **CHECKPOINT** — `winget --version` returns a version, and you are in an Admin PowerShell (`whoami /groups | findstr /i "S-1-16-12288"` shows High Mandatory Level).

2. Installation Guide

The project ships a **one-shot installer** (`scripts\install-native.ps1`) that installs everything and initialises the database. ~10 minutes the first time.

Step 1 — Clone the repository

```
# In an Admin PowerShell, pick a folder with no spaces in the path:
cd D:\Udhay\projects
git clone https://github.com/udhay8005/Inventory-management.git Inventory_mngt
cd Inventory_mngt
```

No Git yet? `winget install Git.Git`, then close/reopen PowerShell.

Step 2 — Create your environment file

```
copy .env.example .env
notepad .env
```

In `.env`, **set a strong** `DB_PASSWORD` (the local PostgreSQL `odoo` user's password) and `BACKUP_PASSPHRASE` (24+ chars — encrypts backups; without it backups can't be restored). Leave `ODOO_USER` / `ODOO_USER_PASSWORD` as the placeholders unless you intend to run the optional out-of-process AI worker. Save and close.

Step 3 — Run the installer

```
scripts\install-native.ps1
```

This (idempotently) installs **PostgreSQL, Python 3.12, wkhtmltopdf** via `winget`; clones **Odoo 19** into `.odoo\`; creates a Python venv in `.venv\`; pip-installs Odoo's dependencies **plus** this project's extras (`statsmodels`, `pandas`, `numpy`, `Pillow`, `reportlab`); creates the `wms` database; writes `config\odoo.native.conf`; and runs Odoo's first-time init.

Useful flags:

```
scripts\install-native.ps1 -SkipWinget      # already have PG/Python/wkhtmltopdf
scripts\install-native.ps1 -DbPort 1088     # PostgreSQL on a non-default port
scripts\install-native.ps1 -Reset           # wipe .odoo\ + .venv\ and reinstall (DB kept)
```

Step 4 — Verify the installation

```
Test-Path .odoo\odoo-bin      # True (Odoo source)
Test-Path .venv\Scripts\python.exe # True (venv)
Test-Path config\odoo.native.conf # True (config written)
Get-Service postgresql-x64-*  # Status = Running
```

📌 CHECKPOINT — all four are present and PostgreSQL is **Running**. 📌 CAPTURE — the installer's final "Done" summary.

Dependency policy: `requirements.txt` pins `numpy<2` and `pandas<3` on purpose (Odoo 19 + `statsmodels` compatibility). Do not bump those without testing — see the project's Dependabot policy.

3. Database Setup Guide

The installer already created the database; this phase is **verification** and understanding what exists.

What the installer set up - A PostgreSQL **Windows service** (`postgresql-x64-16/17`) that auto-starts on boot. - A database login role `odoo` (password = your `.env` `DB_PASSWORD`). - A database named `wms`, owned by `odoo`. - Connection details written into `config\odoo.native.conf` (`db_host`, `db_port`, `db_user`, `db_password`).

Verify the connection

```
# Read the real port + user from the config:
Select-String config\odoo.native.conf -Pattern '^db_(host|port|user|name)\s*='

# Connect (replace 1088 with YOUR db_port from above; default is 5432):
$env:PGPASSWORD = (Select-String config\odoo.native.conf -Pattern '^db_password\s*=\s*(.+)$').Matches.Groups[1].Value.Trim()
& "C:\Program Files\PostgreSQL\17\bin\psql.exe" -U odoo -h localhost -p 1088 -d wms -c "SELECT current_database(), version()"
```

Expected output: one row showing `wms` and the PostgreSQL version banner.

```
# Confirm the WMS tables exist (after you install the addons in Phase 7):  
& "C:\Program Files\PostgreSQL\17\bin\psql.exe" -U odoo -h localhost -p 1088 -d wms -c "\dt wms_*" | Select-Object -First 15
```

🔍 CHECKPOINT — `psql` connects and `SELECT` returns a row. 📺 CAPTURE — the successful `psql` connection.

Backups are configured later (Phase 13). For now, just know the DB is a normal PostgreSQL database you back up with the project's encrypted scripts — never by copying the data folder while the service runs.

4. First Launch Guide

Step 1 — Start Odoo (foreground, for the first run)

```
scripts\start-native.ps1
```

Leave this window open. Odoo logs scroll by; wait for a line like `HTTP service (werkzeug) running on ... :8069`.

Step 2 — Verify the service is listening

Open a **second** PowerShell:

```
Get-NetTCPConnection -LocalPort 8069 -State Listen # OwningProcess = python
```

Step 3 — Verify the logs

```
Get-Content .runtime\logs\odoo-native.log -Tail 30 # no CRITICAL/ERROR lines
```

Step 4 — Verify the health endpoint

```
(Invoke-WebRequest http://localhost:8069/wms/health -UseBasicParsing).Content
```

Expected: `{"status":"HEALTHY","db_reachable":true, ...}` (before backups are scheduled you may see `DEGRADED` /warnings about backup age — that's normal until Phase 13).

Step 5 — Open the browser & first login

1. Browse to <http://localhost:8069>.
2. Sign in: `admin` / `admin`.
3. You'll change this password immediately in Phase 6.

🔍 CHECKPOINT — you reach the Odoo home screen as `admin`. 📺 CAPTURE — the Odoo home screen + the `/wms/health` JSON.

Common startup issues | Symptom | Fix | `port 8069 already in use` | Another Odoo is running. `scripts\stop-native.ps1`, or find it: `Get-NetTCPConnection -LocalPort 8069`. | `connection refused to DB` | PostgreSQL service stopped, or wrong `db_port`. `Start-Service postgresql-x64-*`; check the port in the conf. | `password authentication failed` | `.env DB_PASSWORD` $\neq$ the `odoo` role's password. `scripts\reset-pg-password.ps1` (Admin). | Browser shows nothing | Wait for the "running on :8069" log line; Odoo takes 10–30 s to boot. |

5. Windows Service Guide

Running Odoo as a service makes it **start on boot** and **restart on failure** — required for production.

Step 1 — Install the service (installs NSSM automatically)

```
# Admin PowerShell. Approve the single UAC prompt.  
scripts\install-odoo-service.ps1
```

This installs **NSSM** (via winget if needed) and registers an auto-starting, restart-on-failure Windows service named `Odoo-WMS` that runs the proven `start-native.ps1` launcher, depends on PostgreSQL, and writes rotating logs to `.runtime\logs\`.

Step 2–4 — Auto-start & auto-restart are configured for you

The script sets `SERVICE_AUTO_START`, NSSM "restart on any exit (5 s delay)", **and** Windows-native `sc.exe` failure actions as a second safety net.

Step 5 — Verify

```
Get-Service Odoo-WMS           # Status Running, StartType Automatic
(Invoke-WebRequest http://localhost:8069/wms/health -UseBasicParsing).Content
```

Manage it with `Start-Service` / `Stop-Service` / `Restart-Service Odoo-WMS` (Or `services.msc`).

🔍 CHECKPOINT — Odoo-WMS is **Running / Automatic** and `/wms/health` answers.

Upgrading later (apply new code safely)

After you `git pull` new code, apply it to the live DB with the backup-first upgrade script (it backs up, stops the service, runs the module upgrade, restarts, and re-checks health):

```
scripts\upgrade-service.ps1      # all WMS modules
scripts\upgrade-service.ps1 -Modules wms_barcode # a specific module
```

🔍 CAPTURE — `Get-Service Odoo-WMS` showing Running/Automatic.

6. Security Setup Guide

Do **all** of these before go-live.

1 — Change the admin password (UI)

Top-right avatar → **Preferences** → **Account Security** → change password from `admin` to a strong unique value. Or do it together with the keeper in step 2.

2 — Set strong unique passwords for all logins (script)

```
scripts\set-user-passwords.ps1 -Users "admin,storekeeper"
```

Generates 20-char cryptographically-random passwords via the Odoo ORM (properly hashed), and **prints each once to your console — copy them into your password manager immediately**. They are never written to disk.

3 — Confirm the database manager is disabled

The config ships with `list_db = False`, and the project blocks the `/web/database/*` manager pages **and** the destructive POST routes (create/drop/restore/backup). Verify:

```
Select-String config\odoo.native.conf -Pattern '^list_db' # list_db = False
(Invoke-WebRequest http://localhost:8069/web/database/manager -UseBasicParsing -MaximumRedirection 0 -ErrorAction SilentlyCo
```

4 — (Optional) Lock the health endpoint behind a token

`/wms/health` is open by default (so monitors can poll it). To require a token, set the parameter in **Settings** → **Technical** → **System Parameters**: `wms_reports.health_token = <your-secret>`, then poll `http://localhost:8069/wms/health?token=<your-secret>`.

5 — Verify backups & health are real

After Phase 13, `/wms/health` must report `HEALTHY` with a recent `last_backup_age_hours`.

🔍 CHECKPOINT — admin/keeper passwords rotated + stored; `list_db=False`; DB-manager pages blocked. 🔍 CAPTURE — the one-time password output (then store securely & clear screen).

7. Warehouse Configuration Guide

Step 0 — Install the seven addons (once)

Apps menu → remove the "Apps" filter → search `wms` → **Install** in this order (dependencies pull the rest, but installing in order is safest): 1. `wms_location` 2. `wms_fifo` 3. `wms_barcode` 4. `wms_repair_damage` 5. `wms_ai_forecast` 6. `wms_reports` 7. `wms_training`

📷 **CAPTURE** — the Apps screen with all seven **Installed**.

The storage hierarchy

```
Warehouse (the building – created by Odoo on install)
├── Zone      e.g. "East", "Cold Store"      (an area)
│   ├── Rack  e.g. "R01"                    (a physical rack)
│   │   ├── Compartment e.g. shelfxcolumn cell (can span shelves; L/T/U shapes OK)
│   │   │   ├── Slot      the storable unit   (holds stock)
│   │   └── Floor Zone    e.g. "F-01" – bulk floor area, no rack (for pallets/sacks)
```

Create the structure (use the generators — fastest)

WMS → **Configuration** has three wizards: - **Rack Generator** — give a Rack code (e.g. `R01`), number of shelves & columns, slots per compartment → it builds the rack + all compartments + slots and assigns barcodes automatically. - **Zone Generator** — create a zone and (optionally) a batch of racks under it. - **Floor Zone Generator** — create flat floor areas `F-01...F-0n`.

Example layout (a small trust) | Level | Example | Barcode pattern | |--|--|--| | Zone | `East` | — | | Rack | `R01` (6 shelves × 3 columns) | `R01` | | Compartment | shelf 2, column 1 | `R01-SH02-C01` | | Slot | first slot in that compartment | `R01-SH02-C01-SL01` | | Floor zone | `F-01` | `STOC-F-01` |

Naming conventions - **Zones**: short human area names (`East`, `Pharmacy`, `Cold Store`). - **Racks**: `R##` (`R01`, `R02`). - **Compartments/Slots**: auto-generated `...-SH##-C##-SL##` — don't hand-edit. - **Floor zones**: `F-##`.

📷 **CHECKPOINT** — **WMS** → **Warehouse Map** shows your zone/rack/floors; each slot has a unique barcode. 📷 **CAPTURE** — the Warehouse Map + one Rack's visual grid.

8. User Setup Guide

There are **two kinds of identity** — keep them distinct: - **Odoo login users** (`res.users`) — who signs into the app. - **Store Keeper roster** (`WMS` → `Configuration` → `Store Keepers`) — the human names keepers pick from at scan time for the audit trail (not logins).

Create the roster first

WMS → **Configuration** → **Store Keepers** → **New** — add each real person (Ramesh, Lakshmi, Suresh...). These names appear in every scan/damage form's "Store Keeper on duty" field.

Create login users

Settings → **Users & Companies** → **Users** → **New**:

Role	Group to assign	Can do
Admin / Manager	<code>WMS / Manager</code>	Everything — build racks, products, labels, manage roster, run/scrap repairs, all reports.
Store Keeper	<code>WMS / Store Keeper</code> + the capability groups they need: <code>Scan Receipt</code> + <code>Scan Return</code> , <code>Scan Issue</code> , <code>File damage</code> , <code>Submit audits</code>	Scan in/out, file damage, create repair orders, view reports. Cannot edit racks/products or run repairs.
Read-only	<code>WMS / Store Keeper</code> with no capability groups	View reports + warehouse map only (no stock-moving actions).

Why capabilities matter: the scan/damage/audit actions are enforced at the data layer by these capability groups — a keeper without "Scan Issue" cannot issue stock even via the API. Assign exactly what each person needs.

A common setup is **one shared storekeeper login per shift**; each keeper picks their own name from the roster, so the audit trail still records the individual.

📷 **CHECKPOINT** — admin can build/manage; the keeper login sees the **WMS** menu + scan wizards but **not** the raw Inventory app or rack editing. 📷 **CAPTURE** — the keeper user's group assignment.

9. Label Printing Guide

The WMS prints **4×1 inch (100×25 mm) direct-thermal** labels: logo / title / SKU / inline barcode, gap-sensor aware.

1. **Install the printer** in Windows (vendor driver). Set the stock to **100 mm × 25 mm, gap media**. Print a Windows test page first.
2. **Confirm the label layout: WMS → Configuration → Label Config** — the layout is admin-configurable (logo width vs content split). Defaults suit True-Ally 100×25 stock.
3. **Calibrate the gap sensor** on the printer (e.g. TSC TE244: run auto- calibration so it finds the die-cut gap) so each label advances exactly one ticket. See `docs/LABEL-PRINTING.md`.
4. **Print a test label:** open any product → **Print** → **WMS Thermal Label** (or from the Product Onboarding wizard). The PDF goes to your default/most- recent printer.
5. **Verify the barcode:** scan the printed label with your handheld — it must read back the product/SKU exactly (focus a scan-wizard field and scan).

📄 CHECKPOINT — a printed label scans back to the right product, one label per ticket (no drift). 📄 CAPTURE — a printed label next to its on-screen preview.

10. Training Guide

The system trains its own users — **WMS → Help & Training**.

- **Help Center / Training Library** — searchable SOP articles for every workflow (receive, issue, return, damage, repair, audit, reports), with screenshots and step lists.
- **Guided Tours** — role-based click-through tours that walk a new user through a real screen ("Getting started", "Scan a receipt", etc.).
- **Visual Academy** — annotated screen-maps and SVG workflow diagrams that show exactly which button does what.
- **Beginner Mode** — on by default per user (under **Preferences**); adds extra hints and demands a **confirmation on the irreversible Scrap action**. Users switch it off once comfortable.

Recommended path for a new employee (first day) 1. Sign in → **Help & Training** → **Getting Started** tour. 2. Read the **Receive / Issue / Return** SOP articles. 3. Do a supervised **practice receipt** (Phase 12) with Beginner Mode on. 4. Follow `docs/15-onboarding-script.md` (first-day script) with their manager.

📄 CHECKPOINT — a new keeper can find Help & Training and complete the "Getting Started" tour unaided. 📄 CAPTURE — the Help & Training landing page.

11. Product Onboarding Guide

Use the guided wizard so every product gets a SKU, barcode, and label in one go.

1. **WMS → Products → Onboard Product** (the `wms.product.onboard` wizard).
2. Enter **name**, **Kind** (Raw / Packaging / Fluid / Finished Good / WIP / Consumable / Tool / Spare — this drives returnability), **unit of measure**, and (for perishables like medicine/feed/ghee) the **expiry date**.
3. The wizard assigns a **unique SKU / internal reference** and a **barcode** (or accept a scanned one). SKUs/barcodes are enforced unique across the system.
4. **Print the label** from the wizard → stick it on the bin/item.
5. **Verify:** open the product — it has a SKU, a barcode, the right Kind, and (if applicable) an expiry date. Scanning the label finds the product.

Perishables: set `wms_expiry_date` so the **Expiry Alerts** report and the weekly digest can warn you before stock expires (FEFO).

📄 CHECKPOINT — the product exists with unique SKU + barcode; its label scans. 📄 CAPTURE — the completed product form.

12. First Inventory Workflow

End-to-end: receive stock, put it in a slot, and confirm the audit trail.

1 — Receive stock

WMS → Scan Receipt: 1. Pick the **Store Keeper on duty** (roster), enter **Received/Taken by** and **Ordered by** (audit triplet — required). 2. **Scan the product barcode** (or pick it), enter the **quantity**. 3. **Scan/choose the destination slot** (e.g. `R01-SH02-C01-SL01`). 4. **Validate**. The wizard creates a stock move into that slot.

2 — Putaway / slot assignment

Receiving **into a specific slot** is the putaway — the slot you scanned is where the stock now lives. (For bulk, choose a Floor Zone.)

3 — Verify the slot

WMS → Reports → Where is it? (search the product) — it shows the slot and quantity. Or open the slot on the **Warehouse Map**.

4 — Verify inventory

WMS → Reports → Slot Occupancy / Oldest Stock (FIFO) — the new quantity appears; FIFO age starts counting from the receipt date.

5 — Verify the audit trail

Open the resulting **stock picking** (or the product's moves): it carries **Taken by / Ordered by / Store Keeper on duty** plus a chatter note — the same triplet shown in reports.

 **CHECKPOINT** — the product shows the received quantity in the correct slot, and the move records the full audit triplet.  **CAPTURE** — the Scan Receipt confirmation + the "Where is it?" result.

13. Backup & Recovery Guide

1 — Install the scheduled backup tasks

```
# Admin PowerShell. Approve UAC.
scripts\install-backup-tasks.ps1
```

Registers two Windows Scheduled Tasks: - **WMS Daily Backup** — every day 13:00 → encrypted DB dump + filestore zip into `.\backups\`, with retention. - **WMS Weekly Restore Drill** — Sundays 03:00 → decrypts + structurally verifies the latest backup **without touching production**.

2 — Verify backups

```
scripts\backup-native.ps1 # run one now
Get-ChildItem .\backups\*.dump.gpg | Sort-Object LastWriteTime -Desc | Select-Object -First 3
(Invoke-WebRequest http://localhost:8069/wms/health -UseBasicParsing).Content # last_backup_age_hours small, status HEALTHY
```

3 — Run a restore drill (proves the backup is recoverable)

```
scripts\restore-drill.ps1 # cheap TOC verification of the latest backup
scripts\restore-drill.ps1 -DryRun:$false # full restore into a throwaway wms_drill_<ts> DB, then drop it
```

4 — Verify recovery (real restore runbook)

A genuine restore (after disaster) uses:

```
scripts\restore-native.ps1 -BackupPath backups\wms-<timestamp>.dump.gpg
```

See `docs/07-deployment.md` and `docs/18-restore-drill.md`.

 **CHECKPOINT** — a fresh `.dump.gpg` exists; the restore drill **passes**; `/wms/health` shows a recent backup.  **CAPTURE** — the restore-drill "PASS" output + the backups folder listing.

Guard your `BACKUP_PASSPHRASE` (in `.env`). Without it the encrypted backups cannot be restored. Store it in your password manager / sealed envelope.

14. Go-Live Checklist

Tick every box before declaring the system live.

❑ Installation verified	.odoo / .venv / config present; PostgreSQL Running
❑ Database healthy	psql connects to wms; WMS tables present
❑ First launch OK	start-native runs; Odoo home reachable
❑ Windows service	Odoo-WMS = Running / Automatic; restarts on failure
❑ Security done	admin + keeper passwords rotated & stored;
	list_db=False; DB-manager pages blocked
❑ Backups healthy	install-backup-tasks done; a fresh .dump.gpg exists
❑ Health endpoint healthy	/wms/health = HEALTHY, db_reachable, recent backup
❑ Warehouse configured	zones / racks / compartments / slots built;
	warehouse map renders; slots have barcodes
❑ Users created	admin (Manager) + keeper (Store Keeper + caps);
	roster populated
❑ Labels tested	printer calibrated; a printed 4x1 label scans back
❑ Training completed	keepers finished the Getting-Started tour + SOPs
❑ First receipt tested	a real receipt lands in the right slot with audit triplet
❑ Restore drill passed	restore-drill verified the latest backup recoverable
❑ Production deployment approved	sign-off by the trust's responsible person

When every box is ticked, the system is **production-ready**.

15. Troubleshooting Guide

Startup

Symptom	Likely cause → fix
Port 8069 in use	A previous Odoo still running → <code>scripts\stop-native.ps1</code> ; or <code>Stop-Service Odoo-WMS</code> .
Service won't start	Check <code>.runtime\logs\service-err.log</code> ; ensure PostgreSQL is Running; re-run <code>scripts\install-odoo-service.ps1</code> .
Odoo boots then exits	A module/upgrade error → run foreground <code>scripts\start-native.ps1</code> and read the traceback.

Database

Symptom	Fix
connection refused	<code>Start-Service postgresql-x64-*</code> ; verify <code>db_port</code> in the conf matches PostgreSQL's actual port (<code>Select-String "\$env:ProgramFiles\PostgreSQL*\data\postgresql.conf" -Pattern '^port'</code>).
password authentication failed	<code>.env DB_PASSWORD</code> ≠ role password → <code>scripts\reset-pg-password.ps1</code> (Admin), then re-sync the conf.
"database wms does not exist"	Re-run <code>scripts\install-native.ps1</code> (idempotent — it creates the DB if missing).

Printing

Symptom	Fix
Labels print blank / shifted	Printer not calibrated to the 100×25 gap → run the printer's gap auto-calibration; confirm media = 100×25 mm gap.
Two labels per ticket / drift	Wrong stock size in the driver → set exactly 100 mm × 25 mm; re-calibrate.
Barcode won't scan	Print density too low or label too small → raise darkness; verify the barcode value in Label Config .

Permissions

Symptom	Fix
Keeper can't scan issue/receipt	Missing capability group → add <code>Scan Issue</code> / <code>Scan Receipt + Scan Return</code> to their user.
Keeper sees raw Inventory app	They were given a stock group directly → keep them on <code>WMS / Store Keeper</code> only.
"Access Error" creating audit/damage	Missing <code>Submit audits</code> / <code>File damage</code> capability → grant it.

Backup / Recovery

Symptom	Fix
<code>/wms/health</code> warns "backup stale/missing"	Run <code>scripts\backup-native.ps1</code> ; confirm WMS Daily Backup task exists (<code>Get-ScheduledTask "WMS Daily Backup"</code>).

Symptom	Fix
Backup fails: "BACKUP_PASSPHRASE is the placeholder"	Set a real <code>BACKUP_PASSPHRASE</code> (24+ chars) in <code>.env</code> .
Restore drill fails	The latest backup is corrupt/incomplete → take a fresh backup and re-drill; investigate disk space.
Can't restore (no passphrase)	The <code>BACKUP_PASSPHRASE</code> used to create the dump is required — recover it from your password manager.

Where to look

- **Logs:** `.runtime\logs\odoo-native.log`, `.runtime\logs\service-*.log`.
- **Health:** `http://localhost:8069/wms/health`.
- **Deeper runbooks:** `docs/07-deployment.md`, `docs/11-maintenance.md`, `docs/13-operations-playbook.md`, `docs/18-restore-drill.md`.

This guide pairs with the in-app Help & Training academy and the design docs in `docs/`. Keep it with your deployment; update the example paths/ports to match your install (`config\odoo-native.conf` is the source of truth).